1. Assignment Overview

In this assignment, you will build a small but complete JavaScript application that retrieves data from a remote data source (either a MySQL database via a server-side API or a public web API such as the Star Wars API – SWAPI) and displays it in a usable, accessible interface.

You may use the Udemy course example as a reference, but you must create your own unique application. Do **not** submit the instructor's sample project from the video course.

The focus is on:

- Writing modern, modular JavaScript
- Consuming data from APIs or databases
- Following contemporary architecture and workflow practices
- Ensuring accessibility and usability

2. Course Objectives and Learning Outcomes

This assignment supports the following course objectives from the syllabus:

- Develop advanced JavaScript code
- Compare contemporary JavaScript technologies and their appropriate use
- Utilize a popular JavaScript framework or library (optional but encouraged)
- Utilize data flow, architecture, persistence, workflow, and version control used in modern applications
- Develop a complex client-side/server-side web application using contemporary practices, utilities, and a framework
- Create server-side web application components using contemporary languages and technologies

3. Technical Requirements

3.1 Data Source Options

Your application must retrieve and display data from **one** of the following:

1. **MySQL Database (Recommended for Full-Stack Practice)**
    - Create a simple database and table(s) in MySQL (e.g., `students`, `books`, `movies`, `tasks`).
    - Build a small server-side API (for example using Node.js with Express, PHP, or another language approved for the course) that returns JSON.
    - Consume that API from your JavaScript front end using `fetch()` or a similar modern HTTP client.
2. **Public Web API (e.g., SWAPI)**
    - You may use the Star Wars API (SWAPI) or another instructor-approved public API.
    - You must design your own interface and data handling logic; do not copy the Udemy instructor's app.
    - Focus on clean data retrieval, error handling, and meaningful presentation of the data.

3.2 JavaScript Requirements (Client Side)

Your client-side code should demonstrate modern JavaScript practices:

- Use ES6+ features such as `const`, `let`, arrow functions, and modules where appropriate.
- Use `fetch()` or `async`/`await` for asynchronous requests.
- Separate logic into functions or modules (e.g., data service, UI rendering, event handlers).
- Update the DOM dynamically based on the data retrieved.
- Handle loading states, empty states, and error states in the UI.

3.3 Frameworks and Libraries (Optional but Encouraged)

You may use a modern JavaScript framework or library, such as:

- React
- Vue.js
- Svelte
- Angular
- Lightweight libraries for HTTP (e.g., Axios) or state management

If you choose a framework, keep the scope small and focused. The goal is to demonstrate understanding of:

- Component-based architecture
- State and props (or equivalent concepts)
- Data fetching in lifecycle hooks or composables

Using plain (vanilla) JavaScript is also fully acceptable and can be a great way to deepen your understanding of the fundamentals.

3.4 Server-Side Requirements (If Using MySQL)

If you choose the MySQL option, your server-side layer should:

- Expose at least one **read** endpoint (e.g., `GET /api/items`).
- Return JSON data with appropriate HTTP status codes.
- Use parameterized queries or an ORM to avoid SQL injection.
- Organize code into separate files for routes, controllers, and database access where reasonable.

Example technologies that are current and widely used:

- Node.js with Express and a MySQL driver or ORM (e.g., `mysql2`, Sequelize, Prisma)
- PHP with PDO or Laravel (if allowed by the course)
- Other server-side stacks approved by your instructor

3.5 Important: Do Not Include `node_modules`

Do NOT include the `node_modules` folder in any of your submissions. This directory is very large and can be recreated at any time using:

```
npm install
```

Instead, ensure that your project includes a complete and accurate `package.json` (and `package-lock.json` or `pnpm-lock.yaml`/`yarn.lock` if applicable).

4. Application Design and Features

4.1 Minimum Feature Set

Your application should include at least:

- A clear, accessible user interface with headings, labels, and meaningful text.
- At least one interactive feature that triggers data retrieval (e.g., a button click, search form, or filter selection).
- Display of multiple records (e.g., a list, table, or cards) from the data source.
- Some way to focus on or view details of a single record (e.g., clicking a list item to see more information).
- Basic client-side validation for any forms or user input.

4.2 Usability and Accessibility

Design your UI so it is usable and accessible to a wide range of users. At a minimum:

- Use semantic HTML5 elements (`<main>`, `<section>`, `<article>`, `<header>`, `<nav>`, `<footer>`).
- Ensure headings are in logical order (`h1` once, then `h2`, `h3`, etc.).
- Provide descriptive text for buttons and links (avoid “Click here”).
- Make sure interactive controls are keyboard accessible (using native elements like `<button>` and `<a>` where possible).
- Use ARIA attributes only when necessary (for example, `aria-live` for dynamic status messages).

Example: announcing loading state for screen readers:

```
<div id="status" aria-live="polite">Ready.</div>

<script>
  async function loadData() {
    const statusEl = document.getElementById('status');
    statusEl.textContent = 'Loading data…';
    try {
      const response = await fetch('/api/items');
      if (!response.ok) throw new Error('Network error');
      const items = await response.json();
      statusEl.textContent = `Loaded ${items.length} items.`;
      // render items...
    } catch (error) {
      statusEl.textContent = 'Failed to load data.';
      console.error(error);
    }
  }
</script>
```

4.3 Example Project Ideas

You are encouraged to choose a topic that interests you. Some examples:

- Star Wars data explorer using SWAPI (e.g., browse people, planets, or starships).
- Course list viewer that pulls course data from a MySQL database.
- Simple task manager that loads tasks from a database and displays them with filters.
- Book or movie catalog with search and detail views.
- Student club directory with members and events retrieved from a database or API.

5. Example Architectures and Code Snippets

5.1 Client-Side Fetch Example (Public API)

The following example shows how to retrieve data from SWAPI and display a simple list of Star Wars people:

```
<main>
  <h1>Star Wars People</h1>
  <button id="loadPeople">Load People</button>
  <div id="status" aria-live="polite">Ready.</div>
  <ul id="peopleList"></ul>
</main>

<script>
  const loadButton = document.getElementById('loadPeople');
  const peopleList = document.getElementById('peopleList');
  const statusEl = document.getElementById('status');

  loadButton.addEventListener('click', loadPeople);

  async function loadPeople() {
    statusEl.textContent = 'Loading people…';
    peopleList.innerHTML = '';

    try {
      const response = await fetch('https://swapi.dev/api/people/');
      if (!response.ok) {
        throw new Error(`HTTP error! status: ${response.status}`);
      }

      const data = await response.json();
      const people = data.results;

      if (!people.length) {
        statusEl.textContent = 'No people found.';
        return;
      }

      people.forEach(person => {
        const li = document.createElement('li');
        li.textContent = `${person.name} – Height: ${person.height}`;
        peopleList.appendChild(li);
      });

      statusEl.textContent = `Loaded ${people.length} people.`;
    } catch (error) {
      console.error(error);
      statusEl.textContent = 'Error loading people. Please try again.';
    }
  }
</script>
```

5.2 Basic Node.js + Express + MySQL Example

The following is a simplified example of a Node.js API endpoint that returns rows from a MySQL table called `books`. This is for illustration; adapt it to your own schema and environment.

```
// server.js
import express from 'express';
import mysql from 'mysql2/promise';

const app = express();
const PORT = process.env.PORT || 3000;

const pool = mysql.createPool({
  host: 'localhost',
  user: 'your_mysql_user',
  password: 'your_mysql_password',
  database: 'your_database_name',
  waitForConnections: true,
  connectionLimit: 10
});

app.get('/api/books', async (req, res) => {
  try {
    const [rows] = await pool.query('SELECT id, title, author, year FROM books');
    res.json(rows);
  } catch (error) {
    console.error('Database error:', error);
    res.status(500).json({ error: 'Internal server error' });
  }
});

app.listen(PORT, () => {
  console.log(`Server listening on http://localhost:${PORT}`);
});
```

Example client-side code to consume this API:

```
async function loadBooks() {
  const response = await fetch('/api/books');
  if (!response.ok) throw new Error('Network error');
  const books = await response.json();
  // render books to the page...
}
```

5.3 Suggested Project Structure

A small but organized project might use a structure like:

```
project-root/
  package.json
  README.md
  public/
    index.html
    CSS/
      styles.css
    js/
      app.js
      api.js
      ui.js
  server/
    server.js
    db.js
```

- **app.js**: Initializes the app, wires up events.
- **api.js**: Contains functions that call your API (e.g., `getBooks()`).
- **ui.js**: Contains functions that render data to the DOM.
- **server.js**: Express or other server configuration and routes.
- **db.js**: Database connection and queries.

6. Workflow, Tools, and Version Control

6.1 Modern Workflow Practices

As part of this assignment, practice a realistic development workflow:

- Use a modern code editor (e.g., VS Code) with extensions for JavaScript, ESLint, and accessibility checks.
- Run your application locally (e.g., `npm run dev` or `node server/server.js`).
- Test in at least two current browsers (for example, Chrome and Firefox).
- Use browser DevTools (Network tab, Console, Elements) to debug issues.

6.2 Version Control with Git

Use Git to track your progress:

- Initialize a repository with `git init`.
- Create a `.gitignore` file to exclude `node_modules`, build artifacts, and environment files.
- Commit meaningful changes with descriptive messages (e.g., “Add API fetch for books list”).
- Optionally, push to a remote repository (GitHub, GitLab, etc.) for backup and portfolio use.

Example `.gitignore` entries:

```
node_modules/
dist/
.env
.DS_Store
```